

Official Fantasy Tennis invite-from-room: server + client

JFTSE PR 208 investigation, 16-17 August 2026

Document	Invite-from-room current truth
Date	17 August 2026, Europe/Berlin (PT)
Author	Thewind / JFTSE Bot
Client	PE32 FantaTennis.exe, image base 0x400000 (file offset = VA - 0x400000)
Server branch	ThewindMom/JFTSE fix/invite-2349-client-room-id
Tip / base	c92e093a on development 568fc3ec
Upstream PR	sstokic-tgm/JFTSE#208 -- stays closed until Thewind says go
Status	Feature is BOTH a server fix and a client fix. This note supersedes earlier drafts.

How to read this note

Facts marked **live-proven** were observed on the local JFTSE native Java stack (auth 5894, game 5895, relay 5896, chat 5897) with official FantaTennis.exe under Wine, accounts Testmon / Testtwo, room name InviteTest, on 17 August 2026. Facts marked **static RE** come from the PE32 disassembly (image base 0x400000). Facts marked **discarded** were earlier working hypotheses that this investigation ruled out; they are collected in section 9 so they are not re-opened. Early notes that said "roomId=0 means no modal" or "unk0=0 skips 0x138B" are **wrong** and are not current truth.

1. Verdict

The official Fantasy Tennis invite-from-room feature is **both a server fix and a client fix**. Neither side alone produces a seated guest after the host clicks Invite. The server must emit SMSG 0x2349 with the 1-based display id that the official client's channel-list lookup expects, and it must accept the CMSG RoomJoin 0x138B that Yes eventually sends. The stock official client then still fails to join from the Free Channel room list, because Yes state 2 compares the guest's current channel (Chat, id 4) to the invite roomId (Free, id 1) and takes a channel-hop command instead of advancing to state 3. A one-instruction client patch -- NOP of the state-2 mismatch JCC at VA 0x49853c -- is what made an unpoked Yes sit the guest. That patch is **local only**. Do not push FantaTennis.exe.

Server (required)

The required server work is on ThewindMom/JFTSE branch fix/invite-2349-client-room-id, tip c92e093a, based on development 568fc3ec (already a take-over of sstokic-tgm/JFTSE development). The change is surgical: new Invite / Play-with handlers and packets, an invited-player list on Room, and a RoomJoin resolver that accepts both the 0-based internal room id and the 1-based display id when the joining player is on that list. Stefan's Battlemon / guardian work was not overwritten.

SMSG 0x2349 must send **roomId and unk0 as the 1-based display id**. Free Channel is 1. Official Yes looks that id up on the CHANNEL list via 0x466dc0, which compares against the word at channel+0x8a. Sending Room.roomId 0 misses that list; Yes then silent-clears. This is a Yes-path failure, not a show-path failure -- the modal still appears when roomId is 0, which is why the earlier "roomId=0 means no modal" note was wrong.

Official Yes then joins with CMSG RoomJoin 0x138B. The RoomId on that CMSG is the **low word of 0x2349 unk0**, not a separate invented field. 0x4a6e20, the 0x138B writer, skips the send only if that roomId is **signed-negative** (test si, si / j l). It does **not** skip when the value is 0. The earlier comment "unk0=0 skips 0x138B" is wrong and belongs in section 9.

RoomJoin must accept both the 0-based Room.roomId and the 1-based display id when the player is on invitedPlayerIds. Normal room-list joins (player not invited) stay on the exact 0-based match, so the existing room list

is unchanged. Invited GMs must take a visible Free seat, not InvisibleGmSlot 9: the test accounts are GM, and slot 9 greys READY. Finally, do **not** replay 0x2349 on friend-list open (0x1F49) or login. That crashed the guest at EIP=1 because the popup stage is unbuilt on those screens.

Client (required for Yes to join on official FantaTennis.exe)

The official client **does** show the Yes/No modal on the Free Channel room list. That screen is scene 4, and 4 is in the show-set {2, 4, 6, 7, 8, 9, 0xA}. This is live-proven (screenshot 102). The waiting room is scene 5: the host can send Invite from there, but the guest cannot show the box while sitting in a waiting room. Widening the scene list is not the fix, and NOP'ing the scene gate already ran the handler then AV'd the guest on screens where the popup stage is unbuilt.

After Yes, state 2 compares the guest's current channel (0x466e50, word at +0x8a) to the invite roomId stored at invite+0x802. On the room list the guest is on Chat (id 4); the invite room is Free (id 1). Mismatch takes command 0x44 -- a channel / login hop, **not** 0x138B -- and never reaches state 3. That is why a stock official exe plus this server branch shows the box but does not seat the guest. It is a client bug / limitation, not a missing server packet.

The working client patch is a NOP at **VA 0x49853c (file 0x9853c): 75 0f -> 90 90**. That is the state-2 mismatch JCC (jne +0x0f over mov [esi+0x80c], 3). After the NOP, a mismatch falls through into state 3. Live retest 17 August 2026 ~19:30 PT, **no memory poke**: Yes at countdown 6 -> CMSG 0x138B roomId=1 -> SMSG RoomJoin result 0 -> Testtwo seated next to Testmon in InviteTest (screenshots 106 and 107).

The working 0x2349 live hex, captured on the game socket, is:

```
54 00 65 00 73 00 74 00 6D 00 6F 00 6E 00 00 00 01 00 01 00 00 00
UTF-16LE "Testmon" + NUL + short roomId=1 + int32 unk0=1
```

Current truth in one line

Server sends 0x2349 roomId=1 unk0=1 on game 5895; official client shows Yes/No on scene 4; NOP 0x49853c makes Yes fall through to state 3 and emit 0x138B roomId=1; server seats the invited GM on a visible Free slot. Everything else in this note is supporting evidence, addresses, or a dead end.

2. Product facts

The product sequence is short and local on both Yes and No. The host, already in a waiting room (live-proven scene 5), opens the friend-list context menu and clicks Invite. The official client sends CMSG 0x2347 carrying the target name as UTF-16LE plus NUL and a short roomId. Official sends the **1-based display id** -- 1 for the room shown as "1. InviteTest" -- not the server's internal 0-based Room.roomId.

The server acknowledges with SMSG 0x2348 result 0. The host sees the toast "Invitation message sent." That toast is not evidence that the guest received anything; it only confirms 0x2348. Delivery to the guest is a separate SMSG 0x2349 on the **game socket (5895)**, published via Rabbit PACKET_ONLY on game.messenger.friendList (and chat.messenger.friendList if a chat TCP session exists). Game delivery is enough. Chat 5897 often has no guest TCP at all; requiring chat delivery was a discarded hypothesis.

0x2349 is **popup-only**. The handler shows popup type 0x53 with locale MSG_ASK_ACCEPT_INVITE_FRIEND. The visible text is that locale string plus "\n\n" plus the host name. The room id is **not** sprintf'd into the text. The number drawn on the box is the 15-second countdown timer (popup+0x218, which is CStageManager+0x790), not the roomId. Reading that 0 as roomId was a discarded hypothesis.

Yes is local: there is no CMSG 0x234A on the Invite Yes path. No is local as well. 0x234A is the friend-menu **Play with** request, a different product action that happens to share the 0x234x family and the same invite object. After Yes, the client state machine (tick 0x498450) eventually sends CMSG RoomJoin 0x138B. Treating Invite Yes as 0x234A was a discarded hypothesis and produced the wrong writer (0x498673) and the wrong +0x80a flag.

3. Packet family

The 0x234x family is five opcodes. Invite uses 0x2347 / 0x2348 / 0x2349. Play with uses 0x234A / 0x234B. The join that Invite Yes is trying to reach is the existing room-join pair 0x138B / 0x138C. Field lists below are from static RE of the official writers and handlers plus live captures; no fields were invented.

Opcode	Name	Dir	Fields
0x2347	CMSG InviteFriend	C->S	string playerName (UTF-16LE + NUL), short roomId
0x2348	SMSG InviteFriend	S->C	short result (0 success, -2 none, -3 cant)
0x2349	SMSG InviteFriendNotify	S->C	string playerName, short roomId, int32 unk0
0x234A	CMSG PlayWith	C->S	string playerName, short roomId
0x234B	SMSG PlayWith	S->C	short result, string playerName, short roomId, int32 unk0
0x138B	CMSG RoomJoin	C->S	short roomId, byte unk0, optional password
0x138C	SMSG RoomJoin	S->C	char result (0 = seated)

0x2348 result codes

Result 0 is success and is what produces the host toast "Invitation message sent." Result -2 is "none" (target offline, or the host invited themselves). Result -3 is "cant" (not friends, or the host is not in a waiting room). The handler does not invent further codes.

0x234B result codes

Play-with results are a different table and must not be mixed with 0x2348. Result 1 is success and sets invite+0x80a = 1 so that a later state 3 can send 0x234A. Result 0 activates the object with +0x80a = 0. Result -1 is not connected; -2 shows MSG_TOPLAY_CANT; -3 is no game room; -4 is full; -5 is already joined. Invite Yes leaves +0x80a at 0, which is why state 3 takes the 0x138B writer and not the 0x234A writer.

Opcode	Result	Meaning
0x2348	0	Success -- host toast "Invitation message sent."
0x2348	-2	None -- offline or self
0x2348	-3	Cant -- not friends / host not in waiting room
0x234B	1	Success -- sets +0x80a = 1 (later state 3 can send 0x234A)
0x234B	0	Activate with +0x80a = 0
0x234B	-1	Not connected
0x234B	-2	MSG_TOPLAY_CANT
0x234B	-3	No game room
0x234B	-4	Full
0x234B	-5	Already joined
0x138C	0	Seated (live-proven on invited join)

4. Official client map (FantaTennis.exe)

The official client is a PE32 image, 3,801,088 bytes, image base 0x400000. File offset equals VA - 0x400000. Official sha256 5477f0827acae66976403aecd2e9ebffeb4fa28da1fedae5f9541ec25e336c31. The current local patched exe (Yes JCC retargets plus the 0x49853c NOP, plus the existing init+show cave) is sha256 76726d1298a1f327dece9d2b1b5c62051ca24fc122d565c35c53a26928efbbb6.

Dispatch of the 0x2349 notify is the standard opcode compare: cmp edx, 0x2349 / je 0x48d56d / call 0x484d30. 0x234B is handled at 0x484ea0. Both handlers share the invite object and the CStateManager popup, but they set different +0x80a flags and therefore different state-3 writers.

CStageManager and the popup

CStageManager is a singleton at 0x7b33f8 (getter 0x4b0520, ctor 0x4b0440, vtable 0x6F1CD0). The current scene dword lives at +0x56c. The popup is embedded at +0x578. Visibility is popup+0xbc, which is CStageManager+0x634; 0x4ae850 treats a non-zero value as "busy / another overlay is up." The countdown timer is a float at popup+0x218 = CStageManager+0x790. Show goes through 0x4ae9c0 (type 0x53 for invite). Official show of the box itself is at 0x56daea. The init+show cave on the local test exe is still at 0x681aa8.

Invite object

The invite object is at 0x7b2bd0, created lazily through 0x498760. Layout that this investigation actually used:

Offset	Type / width	Role (live-proven or static RE)
+0x000 / +2	wchar name, max 0xC	Host name stored from 0x2349; shown under the locale text
+0x802	word roomId	Copied from 0x2349 roomId; state 2 compares this to current channel +0x8a
+0x804	int32 unk0	Copied from 0x2349 unk0; state 3 passes low word into 0x2D / 0x138B
+0x808	word last channel	Last seen channel +0x8a. Live on failed Yes: 4 (Chat). 0xFFFF = unset
+0x80a	word Yes-flag	0 for Invite Yes (0x138B path). 1 after successful 0x234B (0x234A path)
+0x80c	dword state	State machine: 0 clear, 1 Yes-entered, 2 channel compare, 3 send
+0x810	flag	If non-zero, state 3 writes 0x1389 RoomCreate instead of join / play-with

Live on the failed Yes (countdown 6, before the NOP): guest invite @0x7b2bd0 had active=1, roomId=1, unk0=1, +0x808=4 (Chat), +0x80a=0, +0x80c=2. That single snapshot is the whole state-2 story: the object is correctly armed for Invite Yes, and the only thing blocking 0x138B is the Chat != Free compare.

0x2349 handler gates

Handler 0x484d30 applies three gates before it will parse the packet and show the modal. Any failure jumps to 0x484e85 and the box is not shown. These are static RE, confirmed by the live fact that the box **does** appear on scene 4 when all three pass.

#	Check	Fail behaviour
1	0x4ae850 busy: another overlay is up (CStageManager+0x634 != 0)	jmp 0x484e85, no modal
2	[invite_obj] != 0 -- an accept is already active	jmp 0x484e85, no modal
3	scene +0x56c not in {2, 4, 6, 7, 8, 9, 0xA}	jmp 0x484e85, no modal

If all three pass, the handler parses name / roomId / unk0, stores them through 0x498370, and shows type 0x53 via 0x4ae9c0. Widgets are already bound on a fresh Wine prefix -- there is no extra "open messenger first" step. The official modal is popup type 0x53, not GUIPOP_MESSENGER_INVITE. Messenger is only the friend-list context menu that originates Invite (0x2347) and Play with (0x234A).

Scene sets

Three different scene sets gate three different product actions. They overlap but are not the same, which is why "the guest is on a legal screen" is not the same statement as "the host can click Invite." Live-proven values: waiting room = 5, Free Channel room list = 4.

Action	Scene set	Notes
Show 0x2349 / send Play-with 0x234A	{2, 4, 6, 7, 8, 9, 0xA}	Room list (4) is in. Waiting room (5) is not.
Host Invite 0x2347	{5, 6, 7, 8, 9, 0xA}	Waiting room (5) is in. Room list (4) is not.
Yes state 1 continues	{2, 4, 6, 0xA}	4 or 0xA -> state 2; 2 or 6 -> command 0x54; else clear

State 1's continuation set is the reason a guest who somehow accepted from the wrong screen would never reach the channel compare. On the room list (scene 4) state 1 advances to state 2, which is exactly what we observed live (+0x80c =

2 after Yes at countdown 6).

5. Yes state machine

Per-frame tick is 0x498450, dispatch table at 0x498744. Yes entry is 0x4983c0: it sets [obj]=1, +0x80c=1, +0x80a=arg (0 for Invite Yes), then runs grade check 0x46dfe0. A failed grade check toasts MSG_INVITE_FRIEND_RESULT_GRADE and does not arm the state machine. A click that races the 15 s expiry / 0x56d710 hide can also miss 0x4983c0 entirely -- that is the countdown-5 hide-only event in section 6, where +0x808 stayed 0xFFFF.

State	VA	What it does
0	0x498713	Clear [obj] and +0x80c. Idle / dismiss.
1	0x4984c4	Scene gate {2,4,6,0xA}. 4 or 0xA -> state 2; 2 or 6 -> command 0x54; else clear.
2	0x498510	Compare current channel +0x8a (via 0x466e50) to invite +0x802. Match -> 3. Mismatch -> 0x466dc0 then +0x90(0x44, room*, 0).
3	0x498591	If +0x810 != 0 -> 0x1389 RoomCreate. Else if +0x80a != 0 -> 0x234A Play with. Else -> +0x90(0x2D, &word[invite+0x804], 2) -> 0x138B.

State 2 in detail (the official join blocker)

0x466e50 returns the room / channel the local player is already in. State 2 compares word [that+0x8a] to invite+0x802. On a match it writes +0x80c = 3 and returns. On a mismatch it calls 0x466dc0(invite roomId) to resolve a CHANNEL list entry, then issues +0x90(0x44, room*, 0). Command 0x44 is a channel / login hop. It is **not** 0x138B and it does not seat anyone. Live: Chat id 4 != Free 1 -> 0x44, stuck in the hop, never state 3.

The official JCC at 0x49853c is 75 0f (jne +0x0f) jumping over mov [esi+0x80c], 3. That is the entire official mismatch behaviour. NOP it to 90 90 and the mismatch falls through into state 3. 0x466dc0 itself is not wrong -- it walks the CHANNEL list, not the game-room list, and Free Channel's +0x8a really is 1. A 0x2349 roomId=1 hits Free. A 0x2349 roomId=0 misses, and Yes silent-clears at 0x4983c0 because of je 0x498417 at 0x4983dc, **before** state 1. That is why roomId=0 is a Yes-path bug, not a show-path bug.

State 3 in detail (how 0x138B is actually born)

State 3 is a three-way fork. +0x810 != 0 writes 0x1389 RoomCreate (not used on this path). +0x80a != 0 writes 0x234A through 0x498673 (the Play-with path). The remaining case -- Invite Yes, +0x80a = 0 -- calls +0x90(0x2D, &word[invite+0x804], 2). On the room-list screen, CStageManager+0x90 slot 4 is 0x4abb10, and that slot reaches the 0x138B writer 0x4a6e20. The roomId it sends is the **low word of unk0** (invite+0x804), which is why the server must set unk0 to the same 1-based display id as roomId.

0x4a6e20 skips the send only on a signed-negative roomId (test si, si / jl). Zero is non-negative, so unk0=0 would still emit 0x138B with roomId=0 -- which the server would then have to resolve. The "unk0=0 skips 0x138B" reading of that JCC is a discarded hypothesis. We still send unk0=1, because that is what official uses and what 0x466dc0 / the display id expect.

6. Live proof timeline (17 August 2026, Europe/Berlin)

Accounts: test/test -> Testmon (GM, host), test2/test2 -> Testtwo (GM, guest). Room name InviteTest. Local JFTSE native Java: auth 5894, game 5895, relay 5896, chat 5897. Two Wine prefixes, host brought up first. All times below are Europe/Berlin, labelled PT. Items 1, 2, 5, 6 and 7 are screenshot-backed. Item 4 is a live memory snapshot of the invite object. Item 3 is a negative result (click raced the hide).

1. Official Yes/No modal on the room list (scene 4)

The official client shows the Yes/No modal on the Free Channel room list. The box auto-dismisses in about 15 seconds. Widgets are already bound on a fresh prefix; the messenger window does not need to be open. This single fact retires "Free

Channel room list is outside the show-set" and "need messenger open." Scene 4 is in {2, 4, 6, 7, 8, 9, 0xA}.



Figure 1. Screenshot 102 -- official Yes/No modal on the Free Channel room list (scene 4). Live-proven show path. The number on the box is the 15 s countdown, not the roomId.

2. Working 0x2349: roomId=1, unk0=1

The packet that matches official Yes is 0x2349 with roomId=1 and unk0=1. That is the 1-based display id of Free Channel / "1. InviteTest". 0x466dc0 resolves it to the CHANNEL entry whose +0x8a word is 1. The hex on the wire (game 5895) is the UTF-16LE host name, a NUL, a short 1, and an int32 1.



Figure 2. Screenshot 103 -- 0x2349 delivered with roomId=1 unk0=1. This is the working notify, not a debug poke. Modal is already up on the guest.

3. First Yes at countdown 5 -- hide only

The first Yes click, taken at countdown 5, was hide-only. Invite+0x808 stayed 0xFFFF, which means 0x4983c0 never ran. The click raced the expiry / 0x56d710 hide. This is not a packet bug and not a scene-gate bug. Operational rule, recorded again in section 12: click Yes while the countdown is still high.

4. Yes at countdown 6 -- state 2 Chat mismatch (no 0x138B)

Yes at countdown 6 did enter 0x4983c0. The guest invite object at 0x7b2bd0 was then:

Field	Value	Reading
active / [obj]	1	Yes accepted, object armed
roomId +0x802	1	Free Channel display id from 0x2349
unk0 +0x804	1	Same 1-based id; would be the 0x138B roomId
+0x808 last channel	4	Chat. Guest is on the room list, not in Free.
+0x80a Yes-flag	0	Invite Yes, not Play-with
+0x80c state	2	State 1 advanced (scene 4 -> state 2). Stuck on compare.

State 2's 0x466e50 returned Chat (4), which is not 1, so the tick issued command 0x44 and never 0x138B. This is the official client limitation that the NOP later removes. Forcing state 3 by hand was the experiment that proved state 3's writer was already correct.

5. Force +0x80c = 3 -- 0x138B seats the guest

Poking +0x80c from 2 to 3, with no other field changes, sent CMSG RoomJoin 0x138B roomId=1 unk0=0. The server answered SMSG RoomJoin result 0. Testtwo sat down. That is live proof that (a) unk0=1 is the right 0x138B roomId source, (b) the server's resolveJoinRoom accepts display id 1 for an invited player, and (c) the only missing step on the client was reaching state 3.

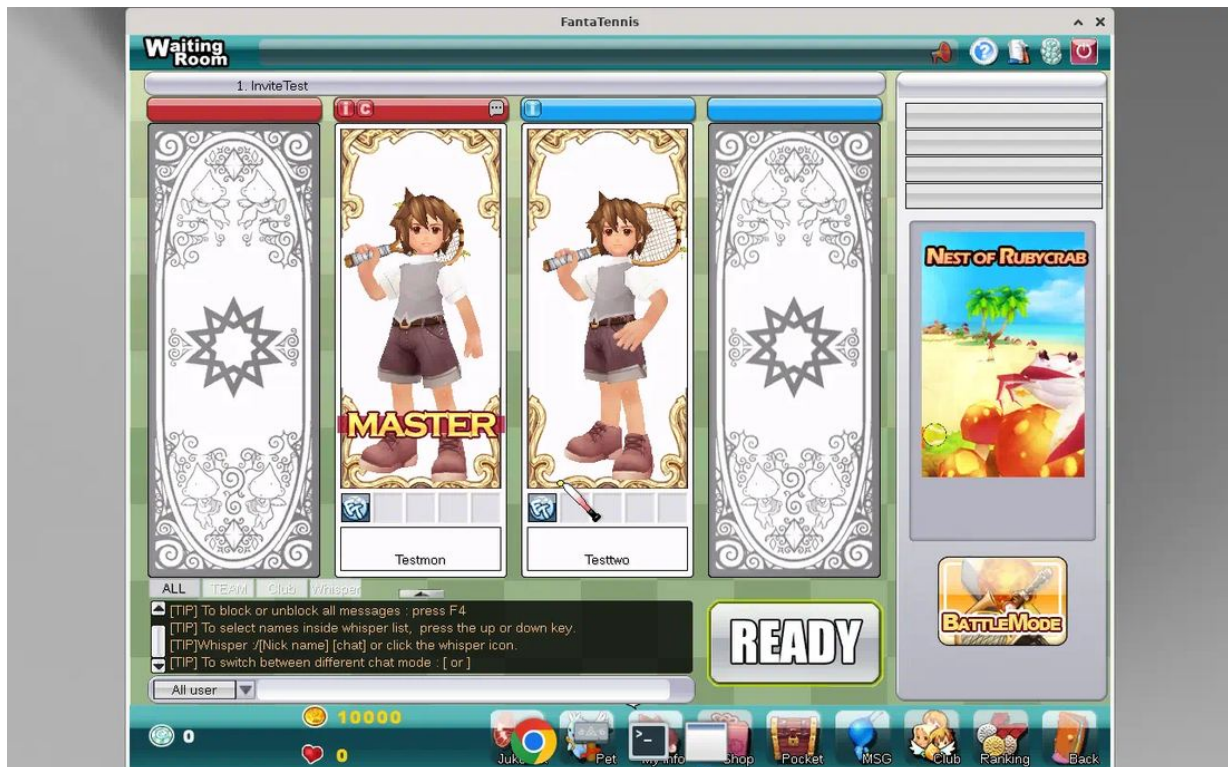


Figure 3. Screenshot 105 -- Testtwo seated after forcing invite+0x80c = 3. Proves the 0x138B / RoomJoin path, not the unpoked Yes path (that is Figure 5).

6. Client patch: NOP the state-2 mismatch JCC

The patch that made unpoked Yes join is one instruction. Backup of the then-current test exe was saved as FantaTennis.exe.yes-state2.

```

VA          0x49853c
File offset 0x9853c      (= VA - 0x400000)
Official    75 0f      jne +0x0f ; skip mov [esi+0x80c], 3 on mismatch
Patched     90 90      nop / nop ; fall through to state 3

```

The local test exe also carries two Yes JCC retargets that were already present from earlier work. They are **not** strictly required to explain the 19:30 PT retest of the state-3 skip alone, but they are on the binary that was retested (the state-3 skip exe = Yes JCC retargets plus this NOP):

VA	File	Bytes	Role
0x4984e1	0x984e1	0f 85 aa 00 00 00	state 1 dismiss -> jne 0x498591 (state 3)
0x49851e	0x9851e	0f 84 6d 00 00 00	state 2 null -> je 0x498591 (state 3)
0x49853c	0x9853c	90 90 (was 75 0f)	THE NOP that made unpoked Yes join

Init+show cave is still at 0x681aa8. Official show remains 0x56daea. Those are pre-existing local test hooks and are not the join fix.

7. Retest ~19:30 PT -- no memory poke

On the patched exe, with no invite-object poke, the host invited from the waiting room and the guest clicked Yes at countdown 6. The client sent CMSG 0x138B roomId=1. The server returned result 0. Testtwo sat next to Testmon in InviteTest, visible, not on InvisibleGmSlot 9. That is the definition of "completely works" in section 10.



Figure 4. Screenshot 106 -- Yes/No at countdown 6 on the patched exe, no memory poke. This is the click that produced 0x138B roomId=1.

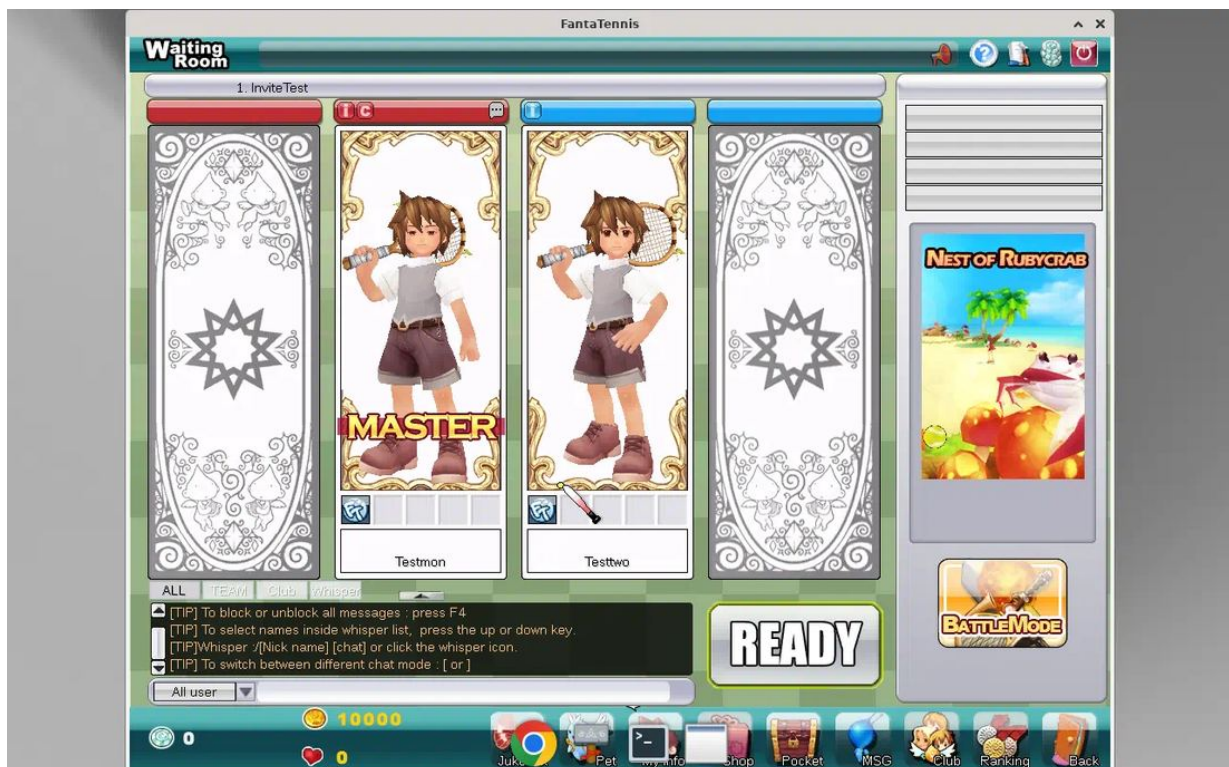


Figure 5. Screenshot 107 -- Testtwo seated next to Testmon in InviteTest after the unpoked Yes. SMSG RoomJoin result 0. Live-proven end state, 17 August 2026 ~19:30 PT.

Current local patched exe sha256 76726d1298a1f327dece9d2b1b5c62051ca24fc122d565c35c53a26928efbbb6. Official (unpatched) sha256 5477f0827acae66976403aecdd2e9ebffeb4fa28da1fedae5f9541ec25e336c31.

7. Server changes on the branch

Branch: <https://github.com/ThewindMom/JFTSE/tree/fix/invite-2349-client-room-id> -- tip c92e093a, base development 568fc3ec (already a take-over of sstokic-tgm/JFTSE development). The work is minimal and does not overlay Battlemon. FriendListRequestHandler was **not** changed; there is no 0x2349 replay on friend-list open or login.

New files

Area	File	Role
game-server	InviteFriendRequestHandler.java	CMSG 0x2347 -> 0x2348 + 0x2349
chat-server	InviteFriendRequestHandler.java	Same handler on chat TCP if present
game-server	PlayWithRequestHandler.java	CMSG 0x234A -> 0x234B
chat-server	PlayWithRequestHandler.java	Same handler on chat TCP if present
server-core	CMSG_InviteFriend (0x2347)	playerName + short roomId
server-core	SMSG_InviteFriend (0x2348)	short result
server-core	SMSG_InviteFriendNotify (0x2349)	playerName + short roomId + int32 unk0
server-core	CMSG_PlayWith (0x234A)	playerName + short roomId
server-core	SMSG_PlayWith (0x234B)	result + playerName + roomId + unk0

Invite handler (game-server and chat-server)

Both invite handlers compute a client-facing display id and write it to both 0x2349 fields. The comment in tree is the 0x466dc0 contract: match channel +0x8a (Free = 1). Do not send room.getRoomId() 0.

```
final short clientRoomId = packet.getRoomId() > 0
    ? packet.getRoomId()
    : (short) (room.getRoomId() + 1);
SMSGInviteFriendNotify.builder()
    .playerName(...)
    .roomId(clientRoomId)
    .unk0(clientRoomId)
```

If the official host already sent the 1-based display id on 0x2347 (live: 1 for "1. InviteTest"), the ternary keeps it. The fallback (room.getRoomId() + 1) is for a client that sent 0 or omitted a usable id, so the notify still carries Free=1 rather than internal 0.

Patched existing files (surgical)

File	Change
Room.java (game + chat)	invitedPlayerIds: ConcurrentLinkedDeque of invited player ids
FTConnection.java (game + chat)	CMSGInviteFriend + CMSGPlayWith added to THREAD_HANDLED_PACKETS
PacketOperations.java	Five opcodes 0x2347-0x234B inserted after S2CFriendsListAnswer
RoomJoinRequestPacketHandler.java (game + chat)	resolveJoinRoom; skip password if invited; invited GMs skip InvisibleGmSlot 9; remove from invited list after seat

resolveJoinRoom prefers the room whose invitedPlayerIds contains the joining player, accepting an exact roomId **or** roomId-1. That is what lets a 0x138B carrying display id 1 sit the guest in internal room 0. If the player is not on any invited list, the function returns the exact 0-based match and the ordinary room-list join is unchanged. After a successful seat the id is removed from the deque so a later ordinary join does not keep the invited privileges (password skip, visible GM slot).

Invited GMs skipping InvisibleGmSlot 9 is not cosmetic. Testmon and Testtwo are GM accounts. Slot 9 greys READY and looks like a broken join even when RoomJoin result is 0. Visible Free seat is part of "completely works."

Commits (oldest first)

Sha	Message
e3cca711	Add official Invite/Play-with handlers and send 0x2349 roomId/unk0 as the 1-based display id
1366d4fa	Add Room.invitedPlayerIds
a10caadc	Add official Play-with handlers
6bec7005	Thread 0x2347/0x234A on game connection
975177eb	Thread 0x2347/0x234A on chat connection
8a682d00	Add opcodes 0x2347-0x234B to PacketOperations
1e85547e	Accept invited 0x138B joins (game) and seat invited GMs visibly
c92e093a	Accept invited 0x138B joins on chat-server

Tip c92e093a is the chat-server twin of 1e85547e. Game-server RoomJoin is not enough on its own if a future guest happens to be joined through chat; the resolver and the visible-GM seat rule have to exist on both servers. No force-push of development. sstokic-tgm/JFTSE#208 stays closed until Thewind says go.

8. Client patches -- what to ship vs what not

Do not push FantaTennis.exe to JFTSE

The client half of this feature is a local PE patch for the official binary. It does not belong in the JFTSE repository, in a PR, or on development. Ship the server branch. Keep the exe on the test prefixes.

The one NOP that made unpoked Yes join

VA 0x49853c / file 0x9853c / 75 0f -> 90 90. Official: jne over mov [esi+0x80c], 3 when the current channel word does not equal invite roomId. After the NOP: fall through to state 3 -> command 0x2D -> 0x4a6e20 -> CMSG 0x138B with roomId = low word of unk0. That is the whole client fix for the room-list Chat != Free case.

Related bytes present on the local test exe, not strictly required for the 19:30 PT retest of the state-3 skip *alone* -- the retest used the state-3 skip exe, which includes the Yes JCC retargets plus the 0x49853c NOP:

VA	File	Instruction	When it fires
0x4984e1	0x984e1	0f85 aa000000 jne 0x498591	State 1 dismiss retarget to state 3
0x49851e	0x9851e	0f84 6d000000 je 0x498591	State 2 null retarget to state 3
0x49853c	0x9853c	90 90 (was 75 0f)	Mismatch fall-through -- required for unpoked Yes

Dead-end client work (do not revive)

Several client experiments were run and discarded. They are listed here so a later session does not treat them as unfinished work.

Experiment	Why it is dead
Global show-guard skip	Hung init. The busy / overlay guard exists because the popup stage is not always built.
Login 0x1F49 -> 0x2349 replay	Crashed the guest at EIP=1. Popup stage unbuilt on login / friend-list open.
NOP the scene list alone	Handler ran, then AV on screens where the popup stage is unbuilt. Scene 4 already shows.
Invent auto-join or a fake letter (0x1F61)	Not the official product. Invite Yes is a popup + RoomJoin, not a letter.
Treat Invite Yes as 0x234A	Wrong writer (0x498673) and wrong +0x80a. 0x234A is Play with.
Add a roomId field to S2C 0x1F4A	Client already uses the existing trailing short. Do not invent a field.

Experiment	Why it is dead
GameManager.getRoomId() start at 1	Room list and ordinary join are 0-based and already work. Display id is a 0x2349 concern only.

9. Dead ends we ruled out

This section is the graveyard for earlier notes that were wrong, and for hypotheses that looked reasonable before the live session. Do not present any of these as current truth. Do not re-open them without new evidence that contradicts the 17 August 2026 captures.

Discarded claim	Current truth
"0x2349 never arrives"	It does, on game 5895. Host toast "Invitation message sent." is 0x2348; the guest packet is a separate 0x2349.
"Need messenger open"	Official modal is popup type 0x53, not GUIPOP_MESSENGER_INVITE. Messenger is the friend-list context menu (Invite 0x2347 / Play with 0x234A).
"roomId=0 means no modal"	WRONG. Modal shows on scene 4 with roomId=1. roomId=0 fails the CHANNEL lookup on Yes (0x4983dc -> 0x498417), not the show.
"unk0=0 skips 0x138B"	WRONG. 0x4a6e20 is a signed-negative skip (test si,si / jl). Zero would still send.
"The 0 on the box is roomId"	It is the 15 s countdown (popup+0x218 / CStageManager+0x790). Room id is not sprintf'd into the locale text.
"Chat must deliver 0x2349"	Game delivery is enough. Chat 5897 often has no guest TCP.
"Free Channel room list is outside the show-set"	It is scene 4, which IS in {2,4,6,7,8,9,0xA}. Official shows the box there (Figure 1).
"Widen the scene list"	Not a simple fix. NOP'ing the gate already ran the handler then AV'd the guest on screens where the popup stage is unbuilt.

Two of those rows -- "roomId=0 means no modal" and "unk0=0 skips 0x138B" -- were written down as working notes during the 16 August pass. They were reasonable readings of incomplete traces. The 17 August session falsified both: the modal is a scene-set + busy + already-active question, and the 0x138B skip is a signed-negative test. Current code and current comments must not repeat them except as dead ends.

10. What "completely works" means

On the **patched local exe** plus this server branch the product loop is complete. Host Invite from the waiting room (scene 5) -> guest on the Free Channel room list (scene 4) sees Yes/No -> Yes while the countdown is still high -> CMSG 0x138B roomId=1 -> SMSG RoomJoin result 0 -> guest seated, visible, READY not grey. That loop was run at ~19:30 PT on 17 August 2026 with no memory poke (Figures 4 and 5).

On a **stock official exe** plus this server branch the guest sees Yes/No on the room list. Yes does not join. State 2 compares Chat (4) to Free (1), issues command 0x44, and never reaches state 3. That is a client bug / limitation, not a missing server packet. Shipping the server branch to players who still run the official binary will give them a working modal and a silent Yes. That is expected and should be documented as such; it is not a reason to hold the server work, and it is not a reason to commit the exe.

sstokic-tgm/JFTSE#208 stays closed until Thewind says go. Do not force-push development. The branch tip c92e093a is the review surface.

11. Addresses cheat sheet

All addresses are virtual addresses in official FantaTennis.exe, image base 0x400000. File offset = VA - 0x400000. The NOP that matters is highlighted in the role column.

VA	Role
0x48d51b	Opcodes dispatch (cmp edx, 0x2349 / je 0x48d56d)
0x484d30	0x2349 handler (gates -> parse -> 0x498370 -> show 0x53)
0x484ea0	0x234B handler
0x484e85	0x2349 fail epilogue (any gate miss, no modal)
0x498370	Store name / roomId / unk0 into the invite object
0x4983c0	Yes entry: [obj]=1, +0x80c=1, +0x80a=arg; grade check 0x46dfe0
0x4983dc	je 0x498417 -- roomId=0 CHANNEL miss silent-clears before state 1
0x498450	Per-frame tick (table 0x498744)
0x4984c4	State 1 (scene gate)
0x4984e1	State 1 dismiss JCC (local retarget 0f85aa000000 -> 0x498591)
0x498510	State 2 (channel compare)
0x49851e	State 2 null JCC (local retarget 0f846d000000 -> 0x498591)
0x49853c	State 2 mismatch JCC -- THE NOP (75 0f -> 90 90), file 0x9853c
0x498591	State 3 (RoomCreate / Play-with / 0x138B fork)
0x498673	0x234A writer (Play-with path, +0x80a != 0)
0x498713	State 0 clear
0x498760	Invite object getter -> 0x7b2bd0
0x4a6e20	0x138B writer (signed-negative skip, not a zero skip)
0x4abb10	Room-list +0x90 slot 4 (reaches 0x4a6e20)
0x4ae850	Busy / overlay check (CStageManager+0x634 != 0)
0x4ae9c0	Popup show (type 0x53 for invite)
0x4aea10	CStageManager +0x90
0x4b0440	CStageManager ctor (vtable 0x6F1CD0)
0x4b0520	CStageManager getter -> 0x7b33f8
0x466dc0	CHANNEL list lookup by +0x8a (Free = 1)
0x466e50	Current channel / room (state 2 compare source)
0x46dfe0	Grade check on Yes
0x56d710	Popup hide / expiry path (raced by a late Yes click)
0x56daea	Official show
0x57002c	Friend-menu Invite 0x2347
0x570101	Friend-menu Play with 0x234A
0x681aa8	Local init+show cave (test exe only)
0x7b2bd0	Invite object (lazy)
0x7b33f8	CStageManager singleton

CStageManager / invite object offsets

Object	Offset	Field
CStageManager	+0x56c	Scene dword (waiting room = 5, Free Channel room list = 4)
CStageManager	+0x578	Embedded popup
CStageManager	+0x634	Visible = popup+0xbc; non-zero = busy

Object	Offset	Field
CStageManager	+0x790	Countdown float = popup+0x218
Invite	+2	wchar name, max 0xC
Invite	+0x802	roomId (word)
Invite	+0x804	unk0 (int32); low word is 0x138B roomId
Invite	+0x808	Last seen channel +0x8a
Invite	+0x80a	Yes-flag (0 Invite / 1 Play-with)
Invite	+0x80c	State
Invite	+0x810	RoomCreate flag

12. Wine / test notes

Short reproduction notes so Stefan can run the same loop without rediscovering the prefix foot-guns. Accounts: test/test -> Testmon (host, GM), test2/test2 -> Testtwo (guest, GM). Room name InviteTest. After every game-server restart reset the login row, or the next connect is rejected as already online:

```
UPDATE Account SET status=0;  
UPDATE Player SET online=0;
```

Never `kill -f FantaTennis`. That pattern matches more than one process and has taken down the wrong prefix. Kill by PID, then `rm pfx.lock` on the prefix you actually stopped. Bring the host up first on a fresh prefix, then the guest. Two first-inits at once hang both clients. Click Yes while the countdown is still high -- countdown 5 already raced the hide once (item 3 in section 6); countdown 6 was reliable.

Expected result on the patched local exe + branch tip c92e093a: host toast "Invitation message sent.", guest Yes/No on the Free Channel room list, Yes at countdown 6, CMSG 0x138B roomId=1, SMSG RoomJoin result 0, Testtwo seated next to Testmon, READY usable. Expected result on a stock official exe + the same branch: modal shows, Yes does not join. That second outcome is the client limitation in sections 5 / 10, not a server regression.

End of note. 17 August 2026, Europe/Berlin (PT). Thewind / JFTSE Bot. Do not push FantaTennis.exe. Do not force-push development. sstokic-tgm/JFTSE#208 stays closed until Thewind says go.